Document number: D1790R0
Date: 2019-06-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to enable better DynamicBuffer composition

Introduction

P1100r0 *Efficient composition with DynamicBuffer* showed how the current specification of the DynamicBuffer type requirements inhibits layered composition of synchronous and asynchronous I/O operations. This new paper captures the LEWGI discussion of P1100r0 at the Kona 2019 meeting, wherein the root cause of the design issue was explored, and an alternative approach was discussed and accepted.

Background

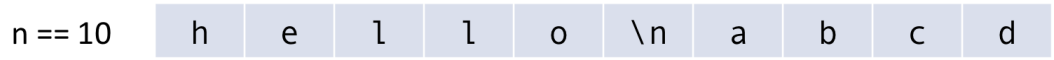
The DynamicBuffer type requirements, and its supporting algorithms and implementations, have the following goals:

- To automatically grow to accommodate data
- To efficiently support delimited protocols
- To be adaptable to support circular buffers

A trivial use of a DynamicBuffer is illustrated in this example:

```
vector<unsigned char> data;
// ...
size_t n = net::read(my_socket,
    net::dynamic_buffer(data, MY_MAX),
    net::transfer_at_least(1));
```

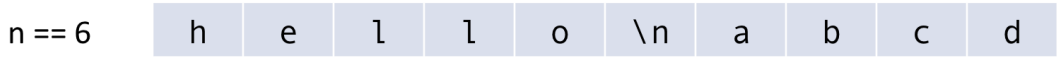
The `net::dynamic_buffer` function creates a DynamicBuffer as a view on to the underlying memory associated with the vector `data`. The vector `data` is then automatically resized to accommodate newly received bytes, and following the `read` operation will contain these bytes.



For delimited protocols, a typical use may look similar to the following example:

```
string data;
// ...
size_t n = net::read_until(my_socket
    net::dynamic_buffer(data, MY_MAX),
    '\n');
```

After `read_until` completes, the vector contains all newly received bytes, while `n` denotes the position of the first delimiter:



Before issuing another `read_until` to obtain the next delimited record, the application should remove the first record from the buffer:

```
data.erase(0, n);
// issue next read_until ...
```

Similarly, use of a DynamicBuffer with a `write` operation will automatically shrink the underlying memory as the data from it is consumed:

```
size_t n = net::write(my_socket,
    net::dynamic_buffer(data));
// data is now empty
```

Both the statically sized buffer sequences, which are created by `net::buffer()`, and the dynamic buffers created by `net::dynamic_buffer`, should be considered as views on to some underlying memory. The difference between the two is that a dynamic buffer will resize the underlying memory as required.

To support this a DynamicBuffer is defined as follows:

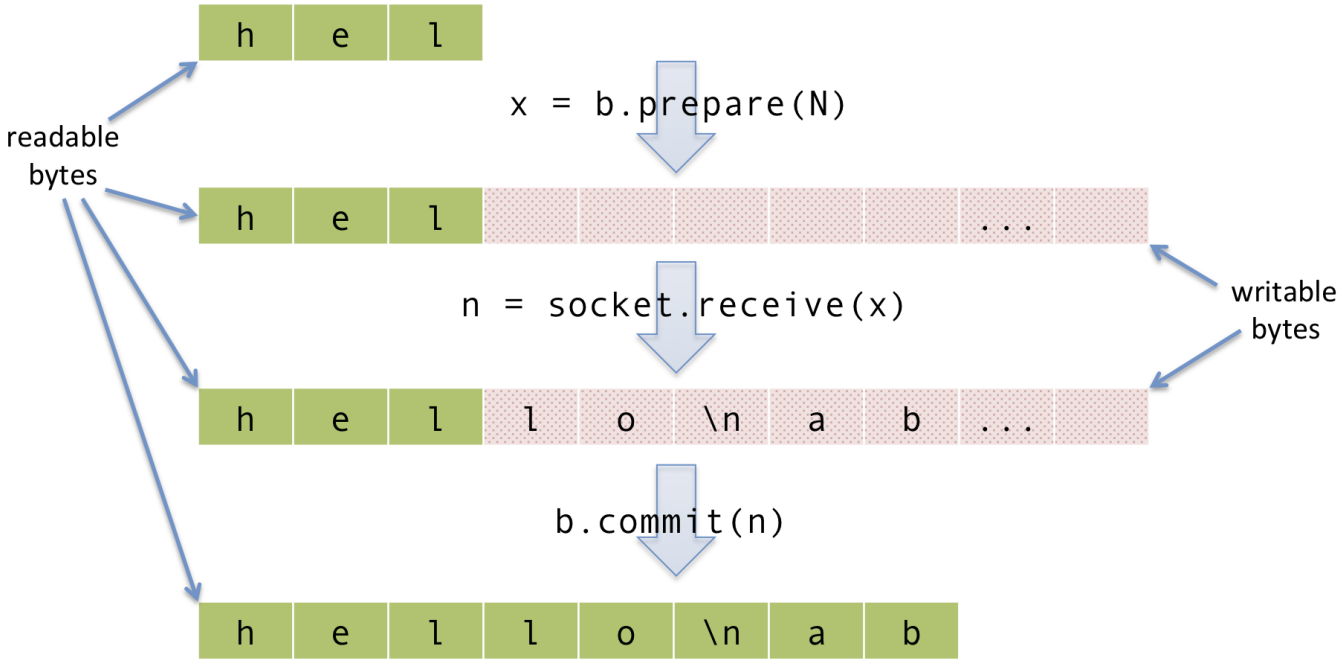
> A *dynamic buffer* encapsulates memory storage that may be automatically resized as required, where the memory is divided into two regions: readable bytes followed by writable bytes. These memory regions are internal to the dynamic buffer, but direct access to the elements is provided to permit them to be efficiently used with I/O operations. [*Note:* Such as the `send` or `receive` operations of a socket. The readable bytes would be used as the constant buffer sequence for `send`, and the writable bytes used as the mutable buffer sequence for `receive`. *–end note*] Data written to the writable bytes of a dynamic buffer object is appended to the readable bytes of the same object.

A DynamicBuffer type is required to satisfy the requirements of `Destructible` and `MoveConstructible`, as well as the requirements shown in the table below.

expression	type	assertion/note pre/post-conditions
`X::const_buffers_type`	type meeting ConstBufferSequence requirements.	This type represents the memory associated with the readable bytes.
`X::mutable_buffers_type`	type meeting MutableBufferSequence requirements.	This type represents the memory associated with the writable bytes.
`x1.size()`	`size_t`	Returns the number of readable bytes.
`x1.max_size()`	`size_t`	Returns the maximum number of bytes, both readable and writable, that can be held by `x1`.
`x1.capacity()`	`size_t`	Returns the maximum number of bytes, both readable and writable, that can be held by `x1` without requiring reallocation.
`x1.data()`	`X::const_buffers_type`	Returns a constant buffer sequence `u` that represents the readable bytes, and where `buffer_size(u) == size()`.
`x.prepare(n)`	`X::mutable_buffers_type`	Returns a mutable buffer sequence `u` representing the

expression	type	assertion/note pre/post-conditions
		writable bytes, and where `buffer_size(u) == n`. The dynamic buffer reallocates memory as required. All constant or mutable buffer sequences previously obtained using `data()` or `prepare()` are invalidated. Throws: `length_error` if `size() + n` exceeds `max_size()`.
`x.commit(n)`		Appends `n` bytes from the start of the writable bytes to the end of the readable bytes. The remainder of the writable bytes are discarded. If `n` is greater than the number of writable bytes, all writable bytes are appended to the readable bytes. All constant or mutable buffer sequences previously obtained using `data()` or `prepare()` are invalidated.
`x.consume(n)`		Removes `n` bytes from beginning of the readable bytes. If `n` is greater than the number of readable bytes, all readable bytes are removed. All constant or mutable buffer sequences previously obtained using `data()` or `prepare()` are invalidated.

This separation between the readable and writable bytes can be visualised in the following sequence of operations:



Historically, the separation of readable bytes and writable bytes originally followed the design model of `std::streambuf`, which divides its memory into input and output sequences. This design was preserved even though DynamicBuffer was ultimately decoupled from concrete streambuf classes.

Problem

Dynamic buffers are intended to be used in compositions, such as an algorithm that reads a sequence of delimited headers from a socket:

```
void read_headers(std::string& data)
{
  size_t n = net::read_until(my_socket, net::dynamic_buffer(data), '\n');
  // process 1st header and consume it
  n = net::read_until(my_socket, net::dynamic_buffer(data), '\n');
  // process 2nd header and consume it
}
```

However, a problem arises when we want to make our algorithm generic across all DynamicBuffer types:

```
template <typename DynamicBuffer>
void read_headers(DynamicBuffer buf)
{
  size_t n = net::read_until(my_socket, std::move(buf), '\n');
  // process 1st header and consume it
  n = net::read_until(my_socket, ???, '\n');
  // process 2nd header and consume it
}
```

As highlighted by P1100r0, we can see here that the current specification of the DynamicBuffer requirements inhibits layered composition of I/O operations. This is a consequence of the type requirements stipulating move-constructibility.

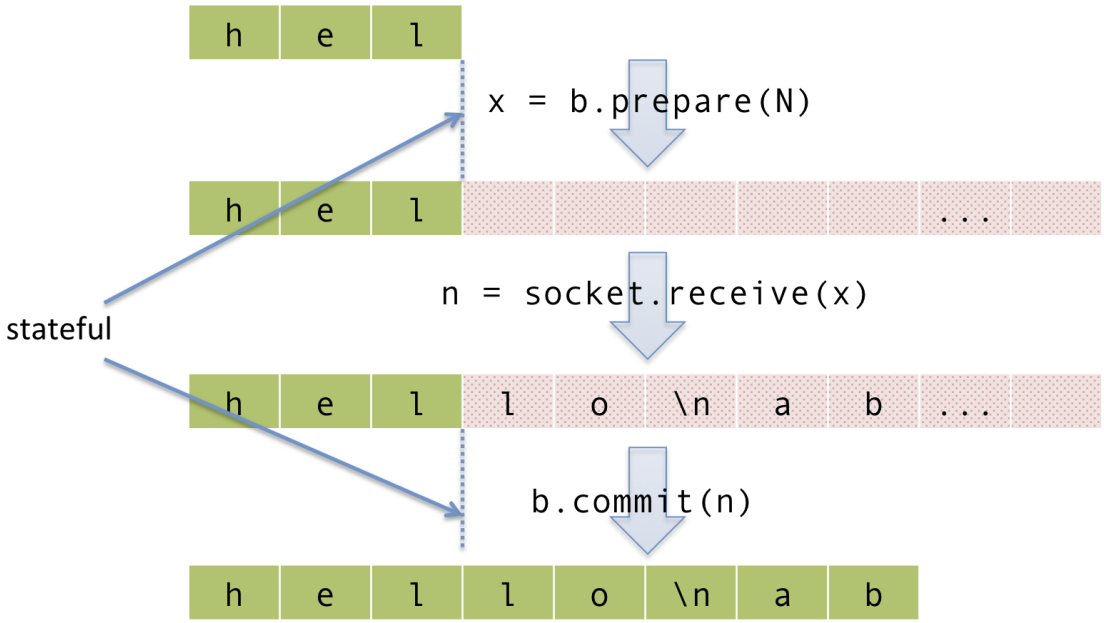
It is worth noting that this problem has no direct impact on the Networking TS itself, as DynamicBuffer is already sufficient for the needs of algorithms defined in the TS). However, we feel it is still worth addressing this problem to enable the development of higher layer algorithms that use dynamic buffers.

Analysis

The DynamicBuffer requirements embody two distinct responsibilities:

1. The ability to dynamically resize underlying memory regions.
2. To separate the buffer into two parts: readable bytes and writable bytes.

It is this second requirement that is the source of the problem, in that it requires a dynamic buffer class to be stateful. Specifically, the dynamic buffer has to maintain state, namely the boundary between the readable and writable regions of memory, across the operations that use it.



This statefulness can be observed in the implementation of concrete dynamic buffers, such as `dynamic_string_buffer`:

```
template <...>
class dynamic_string_buffer
{
  // ...
private:
  basic_string<...>& string_;
  size_t size_;
  const size_t max_size_;
};
```

Solution proposed by P1100r0

P1100r0 proposed removing the `MoveConstructible` requirement from DynamicBuffer, and instead stipulating that DynamicBuffer types be used exclusively by reference.

This changes the typical use of a dynamic buffer as shown below:

```
string data;
// ...
size_t n = net::read_until(my_socket
    net::dynamic_buffer(data, MY_MAX),
    '\n');
```

```
net::dynamic_string_buffer data;
size_t n = net::read_until(my_socket, data, '\n');
std::string s = data.release();
```

but does enable the development of higher level abstractions:

```
template <typename DynamicBuffer>
void read_headers(DynamicBuffer& buf)
{
  size_t n = net::read_until(my_socket, std::move(buf)buf, '\n');
  // process 1st header and consume it
  n = net::read_until(my_socket, buf, '\n');
  // process 2nd header and consume it
}

// ...

net::dynamic_string_buffer data;
read_headers(data);
std::string s = data.release();
```

Alternate solution discussed and accepted by LEWGI

If we consider the way in which these two parts are used within networking TS I/O operations, we see that the distinction between readable and writable parts is actually only important for the duration of an operation.

For example, a DynamicBuffer-enabled read operation:

1. Prepares writable memory to be used as a target for the underlying read operation.
2. Performs the underlying operation.
3. Commits the number of bytes transferred by the operation.

Following the operation, the entire DynamicBuffer consists of readable bytes.

Thus, the alternative solution is to change the DynamicBuffer requirements to have one responsibility only:

- To dynamically resize the underlying memory.

The responsibility for distinguishing between readable and writable bytes is moved to the operations and algorithms that work with DynamicBuffer. This removes the statefulness requirement from DynamicBuffer, and allows DynamicBuffer to be considered a lightweight, copy-constructible type (just as the statically sized ConstBufferSequence and MutableBufferSequence are).

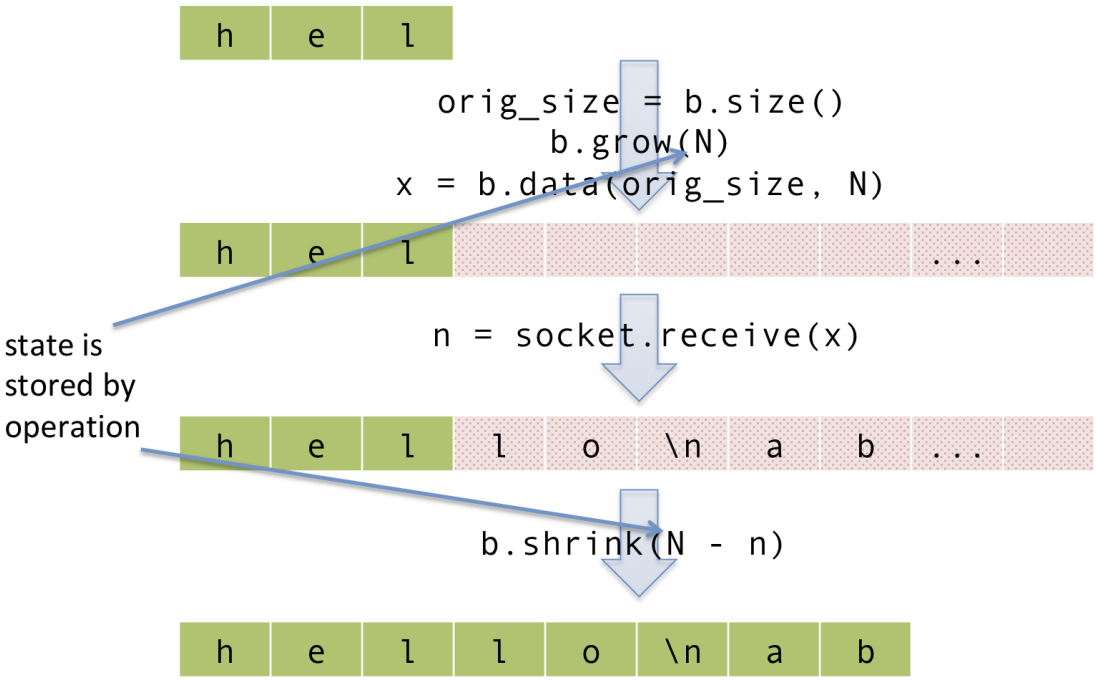
Thus, the DynamicBuffer type requirements are changed from `MoveConstructible` to `CopyConstructible`, as well as the changes shown in the updated table below.

expression	type	assertion/note pre/post-conditions
`X::const_buffers_type`	type meeting ConstBufferSequence requirements.	This type represents the underlying memory ~~associated with the readable bytes~~ as non-modifiable bytes.
`X::mutable_buffers_type`	type meeting MutableBufferSequence requirements.	This type represents the underlying memory ~~associated with the writable bytes~~ as modifiable bytes.

expression	type	assertion/note pre/post-conditions
`x1.size()`	`size_t`	Returns the number of ~~readable~~ bytes.
`x1.max_size()`	`size_t`	Returns the maximum number of bytes~~, both readable and writable,~~ that can be held by `x1`.
`x1.capacity()`	`size_t`	Returns the maximum number of bytes~~, both readable and writable,~~ that can be held by `x1` without requiring reallocation.
~~`x1.data()`~~	~~`X::const_buffers_type`~~	~~Returns a constant buffer sequence `u` that represents the readable bytes, and where `buffer_size(u) == size()`.~~
~~`x.prepare(n)`~~	~~`X::mutable_buffers_type`~~	~~Returns a mutable buffer sequence `u` representing the writable bytes, and where `buffer_size(u) == n`. The dynamic buffer reallocates memory as required. All constant or mutable buffer sequences previously obtained using `data()` or `prepare()` are invalidated. Throws: `length_error` if `size() + n` exceeds `max_size()`.~~
~~`x.commit(n)`~~		~~Appends `n` bytes from the start of the writable bytes to the end of the readable bytes. The remainder of the writable bytes are discarded. If `n` is greater than the number of writable bytes, all writable bytes are appended to the readable bytes. All constant or mutable buffer sequences previously obtained using `data()` or `prepare()` are invalidated.~~
`x1.data(pos, n)`	`X::const_buffers_type`	Returns a constant buffer sequence `u` that represents the region of underlying memory at offset `pos` and length `n`.
`x.data(pos, n)`	`X::mutable_buffers_type`	Returns a mutable buffer sequence `u` that represents the region of underlying memory at offset `pos` and length `n`.
`x.grow(n)`		Adds `n` bytes of space at the end of the underlying memory. All constant or mutable buffer

expression	type	assertion/note pre/post-conditions
		sequences previously obtained using `data()` are invalidated.
`x.shrink(n)`		Removes `n` bytes of space from the end of the underlying memory. If `n` is greater than the number of bytes, all bytes are discarded. All constant or mutable buffer sequences previously obtained using `data()` are invalidated.
`x.consume(n)`		Removes `n` bytes from beginning of the ~~readable~~ bytes. If `n` is greater than the number of ~~readable~~ bytes, all ~~readable~~ bytes are removed. All constant or mutable buffer sequences previously obtained using `data()` ~~or `prepare()`~~ are invalidated.

These new requirements are illustrated by the following sequence of operations:



Concrete dynamic buffer implementations such as `dynamic_string_buffer` now no longer need to maintain the state representing the marker between readable and writable bytes:

```
template <...>
class dynamic_string_buffer
{
  // ...
private:
  basic_string<...>& string_;
```

```
  size_t size_;
  const size_t max_size_;
};
```

Existing dynamic buffer uses are unchanged by this solution:

```
string data;
// ...
size_t n = net::read_until(my_socket
    net::dynamic_buffer(data, MY_MAX),
    '\n');
```

and higher level abstractions are now possible:

```
template <typename DynamicBuffer>
void read_headers(DynamicBuffer buf)
{
  size_t n = net::read_until(my_socket, std::move(buf)buf, '\n');
  // process 1st header and consume it
  n = net::read_until(my_socket, buf, '\n');
  // process 2nd header and consume it
}

// ...

std::string data;
read_headers(net::dynamic_buffer(data));
```

A further advantage of this solution is that, by moving the distinction between the readable and writable bytes from the DynamicBuffer to the algorithms that use it, we now enable algorithms that need to update the notionally “committed” bytes. A simple example of this would be an algorithm that decodes base64-encoded data from a stream-based source: an additional byte of input may require an update to the final byte of output in the buffer. This capability has been requested by Asio users in the past.

Naming

We may wish to consider other options for the names `grow`, `shrink`, and `consume`. The author has no better suggestions to offer at this time.

Implementation experience

The accepted solution was implemented in Asio 1.14.0, which was delivered as part of the Boost 1.70 release.

Proposed changes to wording

In summary, the following changes would be made to the Networking TS wording:

- The DynamicBuffer requirements are changed as shown above.
- The `dynamic_string_buffer` and `dynamic_vector_buffer` classes are altered to satisfy these new requirements.
- The DynamicBuffer algorithms (`read`, `async_read`, `read_until`, `async_read_until`, `write`, and `async_write`) are changed to take DynamicBuffer objects by value, and to use the new `grow` and `shrink` member functions as required.

Detailed wording changes are TBD.